



Soluciones adicionales-ejercicios 20

Estructura de Computadores (Universidad Rey Juan Carlos)

HOJA DE PROBLEMAS. TEMA 20: SEGMENTACIÓN

Soluciones adicionales

Problema 11.- Sea el siguiente código:

```
Etiqueta: SUB $1, $2, $3
           ADD $4, $5, $6
           SUB $5, $4, $8
           ADD $7, $2, $3
           SUB $8, $7, $1
           BNEZ $1, Etiqueta
           LW $9, 0($3)
```

- a) Suponiendo que el procesador segmentado no implementa ninguna técnica de optimización, dibujar el esquema de ejecución de dichas instrucciones:

a.1) Si el salto se toma.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
SUB R1, R2, R3	F	D	E	M	W													
ADD R4, R5, R6		F	D	E	M	W												
SUB R5, R4, R8			F			D	E	M	W									
ADD R7, R2, R3						F	D	E	M	W								
SUB R8, R7, R1							F			D	E	M	W					
BNEZ R1, Etiq										F	D	E	M	W				
SUB R1, R2, R3														F	D	E	M	W

a.2) Si el salto no se toma.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
SUB R1, R2, R3	F	D	E	M	W													
ADD R4, R5, R6		F	D	E	M	W												
SUB R5, R4, R8			F			D	E	M	W									
ADD R7, R2, R3						F	D	E	M	W								
SUB R8, R7, R1							F			D	E	M	W					
BNEZ R1, Etiq										F	D	E	M	W				
LW R9, 0(R3)														F	D	E	M	W

- b) Repetir el apartado a) suponiendo que el procesador segmentado **implementa la técnica de ranura de retardo de tamaño 1.**

Tanto si el salto se toma como si no, en la ejecución de este código la instrucción LW R9, 0(R3) se ejecutará siempre tras la del salto condicional, y después la que corresponda. Esto se puede expresar en una sola tabla, en la que se observa que el cauce, en lugar de 3 ciclos, está parado sólo 2 por culpa del salto:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
SUB R1, R2, R3	F	D	E	M	W													
ADD R4, R5, R6		F	D	E	M	W												
SUB R5, R4, R8			F			D	E	M	W									
ADD R7, R2, R3						F	D	E	M	W								
SUB R8, R7, R1							F			D	E	M	W					
BNEZ R1, Etiqueta										F	D	E	M	W				
LW R9, 0(R3)											F	D	E	M	W			
Instrucción sgte.														F	D	E	M	W

- c) Si el procesador no implementa ninguna optimización, y sin introducir ni eliminar ninguna instrucción, ¿se puede hacer algo para mejorar la eficiencia del código?

Si bien no se pueden eliminar todos los ciclos de parada, se pueden eliminar algunos reordenando el código. Se ofrecen 2 soluciones igualmente válidas:

Solución 1	Solución 2
Etiqueta: ADD R4, R5, R6	Etiqueta: ADD R4, R5, R6
SUB R1, R2, R3	ADD R7, R2, R3
ADD R7, R2, R3	SUB R1, R2, R3
SUB R5, R4, R8	SUB R5, R4, R8
SUB R8, R7, R1	SUB R8, R7, R1
BNEZ R1, Etiqueta	BNEZ R1, Etiqueta
LW R9, 0(R3)	LW R9, 0(R3)

Problema 12.- Sea el siguiente código:

```

LW $1, 0($8)
LW $2, 2($8)
ADD $3, $2, $1
LW $4, 4($8)
LW $5, 8($8)
SUB $6, $5, $4
SUB $7, $6, $3
BNEZ $7, MAYOR
SW $6, 8($8)

```

- d) Suponiendo que el procesador segmentado **no implementa ninguna técnica de optimización**, dibujar el esquema de ejecución de dichas instrucciones. **(1 punto)**

Tanto si se toma el salto como si no se toma, la traza de ejecución de este código vendrá dada por la tabla que se muestra a continuación, en la que “Sgte” será la instrucción **SW \$6, 8(\$8)** o la primera instrucción de la zona de código correspondiente a la etiqueta MAYOR:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
I1	IF	ID	EX	M	W																			
I2		IF	ID	EX	M	W																		
I3			IF			ID	EX	M	W															
I4						IF	ID	EX	M	W														
I5							IF	ID	EX	M	W													
I6								IF			ID	EX	M	W										
I7											IF			ID	EX	M	W							
I8														IF			ID	EX	M	W				
Sgte																				IF	ID	EX	M	W

- e) Repetir el apartado a) suponiendo que el procesador segmentado implementa la técnica predicción de salto como no tomado. **(1 punto)**

Esta técnica supone que el procesador espera que el salto no se tome. Por ello, si el *salto se toma*, la traza de ejecución del código será idéntica a la del apartado a). Sin embargo, si el *salto no se toma*, aplicando esta técnica la traza de ejecución será la siguiente:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
I1	IF	ID	EX	M	W																
I2		IF	ID	EX	M	W															
I3			IF			ID	EX	M	W												
I4						IF	ID	EX	M	W											
I5							IF	ID	EX	M	W										
I6								IF			ID	EX	M	W							
I7											IF			ID	EX	M	W				
I8														IF			ID	EX	M	W	
I9																	IF	ID	EX	M	W

- f) Mejorar todo lo posible la ejecución del siguiente código. Para ello, se podrán utilizar todas las optimizaciones que crean necesarias, considerando que las optimizaciones que deba incorporar el procesador suponen un incremento de P € en el precio del mismo, y que en todo momento se debe mantener un equilibrio entre el coste del procesador y las prestaciones obtenidas. **(1 punto)**

```

LW $1, 0($8)
LW $2, 2($8)
ADD $3, $2, $1
LW $4, 4($8)
LW $5, 8($8)
SUB $6, $5, $4
SUB $7, $6, $3
BNEZ $7, MAYOR
SW $6, 10($8)
J SIGUE
MAYOR: SW $3, 10($8)
SIGUE: ADDI $8, $8, #4

```

Para una mejor comprensión de la solución ofrecida, se analizará este código dividiéndolo en dos partes: la primera incluyendo hasta el BNEZ y la segunda con las instrucciones restantes.

Si se reordena la primera parte del código y se incorpora al procesador la técnica de anticipación, queda como sigue:

	1	2	3	4	5	6	7	8	9	10	11	12
LW \$1, 0(\$8)	IF	ID	EX	M	W							
LW \$2, 2(\$8)		IF	ID	EX	M	W						
LW \$4, 4(\$8)			IF	ID	EX	M	W					
LW \$5, 8(\$8)				IF	ID	EX	M	W				
ADD \$3, \$2, \$1					IF	ID	EX	M	W			
SUB \$6, \$5, \$4						IF	ID	EX	M	W		
SUB \$7, \$6, \$3							IF	ID	EX	M	W	
BNEZ \$7, MAYOR								IF	ID	EX	M	W

En cuanto a la segunda parte del código, estudiaremos el peor caso, aquel en el que no se toma el salto de BNEZ \$7, MAYOR. La traza del código sería la que sigue:

	8	9	10	11	12	13	14	15	16	17	18	19	20	21
BNEZ \$7, MAYOR	IF	ID	EX	M	W									
SW \$6, 10(\$8)					IF	ID	EX	M	W					
J SIGUE						IF	ID	EX	M	W				
ADDI \$8, \$8, #4										IF	ID	EX	M	W

Si se reordena la segunda parte del código, se puede eliminar una instrucción si el procesador incorpora la técnica de ranura de retardo de tamaño 1, quedando el código como sigue y ejecutándose en tan sólo 15 ciclos:

```

LW $1, 0($8)
LW $2, 2($8)
LW $4, 4($8)
LW $5, 8($8)
ADD $3, $2, $1
SUB $6, $5, $4
SUB $7, $6, $3
BNEZ $7, SIGUE
SW $6, 10($8)
SW $3, 10($8)
SIGUE:  ADDI $8, $8, #4

```

Problema 12.- Sea el siguiente código ensamblador:

```

add $2, $2, $14
lw $1, 0($2)
lw $10, 0($7)
sw $10, 0($2)
add $4, $4, $14
lw $3, 0($4)
lw $11, 0($8)
sw $11, 0($4)
add $6, $6, $14
lw $5, 0($6)
lw $12, 0($9)
sw $12, 0($6)
j etiqueta

```

a) Analice su ejecución en el cauce segmentado de MIPS, suponiendo que éste no tiene implementado ningún tipo de optimización, indicando los ciclos de parada que presente y el número total de ciclos que se necesitan para ejecutarlo.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
add \$2, \$2, \$14	IF	ID	EX	M	WB																								
lw \$1, 0(\$2)		IF			ID	EX	M	WB																					
lw \$10, 0(\$7)					IF	ID	EX	M	WB																				
sw \$10, 0(\$2)						IF				ID	EX	M	WB																
add \$4, \$4, \$14									IF	ID	EX	M	WB																
lw \$3, 0(\$4)										IF			ID	EX	M	WB													
lw \$11, 0(\$8)													IF	ID	EX	M	WB												
sw \$11, 0(\$4)														IF			ID	EX	M	WB									
add \$6, \$6, \$14																	IF	ID	EX	M	WB								
lw \$5, 0(\$6)																		IF			ID	EX	M	WB					
lw \$12, 0(\$9)																				IF	ID	EX	M	WB					
sw \$12, 0(\$6)																					IF			ID	EX	M	WB		
j etiqueta																									IF	ID	EX	M	WB

b) Analice su ejecución en el cauce segmentado de MIPS suponiendo que tiene implementada la **técnica de anticipación**, indicando los ciclos de parada que presente y el número total de ciclos que se necesitan para ejecutarlo.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
add \$2, \$2, \$14	IF	ID	EX	M	WB															
lw \$1, 0(\$2)		IF	ID	EX	M	WB														
lw \$10, 0(\$7)			IF	ID	EX	M	WB													
sw \$10, 0(\$2)				IF		ID	EX	M	WB											
add \$4, \$4, \$14						IF	ID	EX	M	WB										
lw \$3, 0(\$4)							IF	ID	EX	M	WB									
lw \$11, 0(\$8)								IF	ID	EX	M	WB								
sw \$11, 0(\$4)									IF		ID	EX	M	WB						
add \$6, \$6, \$14											IF	ID	EX	M	WB					
lw \$5, 0(\$6)												IF	ID	EX	M	WB				
lw \$12, 0(\$9)													IF	ID	EX	M	WB			
sw \$12, 0(\$6)														IF	ID	EX	M	WB		
j etiqueta																IF	ID	EX	M	WB

c) Como diseñador de un procesador MIPS segmentado, se le encarga buscar la forma de optimizar la ejecución del código ensamblador de este problema teniendo en cuenta las siguientes restricciones:

- ☐ Los usuarios carecen de capacidad para reordenar el código por su cuenta.
- ☐ Por cuestiones técnicas, no se puede conectar la salida de la memoria con la entrada de datos de la memoria, lo que afecta a la técnica de anticipación.

En este entorno, evalúe las diferentes técnicas de optimización aplicables y seleccione cuál merece la pena incorporar al procesador MIPS segmentado para ejecutar el código en el menor número posible de ciclos. JUSTIFIQUE ADECUADAMENTE SU RESPUESTA.

Teniendo en cuenta las restricciones que se imponen, mediante la **técnica de anticipación** la ejecución del código sería como sigue

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
add \$2, \$2, \$14	IF	ID	EX	M	WB																		

lw \$1, 0(\$2)		IF	ID	EX	M	WB																		
lw \$10, 0(\$7)			IF	ID	EX	M	WB																	
sw \$10, 0(\$2)				IF			ID	EX	M	WB														
add \$4, \$4, \$14							IF	ID	EX	M	WB													
lw \$3, 0(\$4)								IF	ID	EX	M	WB												
lw \$11, 0(\$8)									IF	ID	EX	M	WB											
sw \$11, 0(\$4)										IF			ID	EX	M	WB								
add \$6, \$6, \$14													IF	ID	EX	M	WB							
lw \$5, 0(\$6)														IF	ID	EX	M	WB						
lw \$12, 0(\$9)															IF	ID	EX	M	WB					
sw \$12, 0(\$6)																IF			ID	EX	M	WB		
j etiqueta																			IF	ID	EX	M	WB	

Si el procesador incorporara técnica de ranura de retardo de tamaño 3, la ejecución del código sería como sigue:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
add \$2, \$2, \$14	IF	ID	EX	M	WB																		
lw \$1, 0(\$2)		IF			ID	EX	M	WB															
lw \$10, 0(\$7)					IF	ID	EX	M	WB														
add \$4, \$4, \$14						IF	ID	EX	M	WB													
lw \$3, 0(\$4)							IF			ID	EX	M	WB										
lw \$11, 0(\$8)										IF	ID	EX	M	WB									
add \$6, \$6, \$14											IF	ID	EX	M	WB								
lw \$5, 0(\$6)												IF			ID	EX	M	WB					
lw \$12, 0(\$9)															IF	ID	EX	M	WB				
j etiqueta																IF	ID	EX	M	WB			
sw \$10, 0(\$2)																	IF	ID	EX	M	WB		
sw \$11, 0(\$4)																		IF	ID	EX	M	WB	
sw \$12, 0(\$6)																			IF	ID	EX	M	WB

Por lo tanto, da igual incorporar cualquiera de estas dos técnicas, dado que ambas hacen que el código se ejecute en 23 ciclos.

Problema 14.- Sea el siguiente código ensamblador, que se va a ejecutar en el cauce segmentado de MIPS:

```

lw $1, 0($14)
add $1, $1, $10
addi $14, $14, 4
lw $2, 0($14)
add $2, $2, $11
beq $1, $2, ETIQUETA
addi $14, $14, 4
lw $3, 0($14)
sub $1, $1, $3
j SEGUIR
ETIQUETA: addi $14, $14, 4
          lw $3, 4($14)
          sub $1, $1, $3
SEGUIR:   sw $1, 8($14)

```

a) Analizar su ejecución, indicando cuántos ciclos de reloj se invierten, sabiendo que el procesador segmentado de MIPS del que se dispone **no implementa ningún tipo de optimización**.

1. Si no se toma el salto condicional:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	
lw \$1, 0(\$14)	IF	ID	EX	M	WB																													
add \$1, \$1, \$10		IF			ID	EX	M	WB																										
addi \$14, \$14, 4					IF	ID	EX	M	WB																									
lw \$2, 0(\$14)						IF			ID	EX	M	WB																						
add \$2, \$2, \$11									IF			ID	EX	M	WB																			
beq \$1, \$2,												IF			ID	EX	M	WB																
addi \$14, \$14, 4																		IF	ID	EX	M	WB												
lw \$3, 0(\$14)																			IF			ID	EX	M	WB									
sub \$1, \$1, \$3																					IF			ID	EX	M	WB							
j SEGUIR																									IF	ID	EX	M	WB					
sw \$1, 8(\$14)																														IF	ID	EX	M	WB

2. Si se toma el salto condicional:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
lw \$1, 0(\$14)	IF	ID	EX	M	WB																										
add \$1, \$1, \$10		IF			ID	EX	M	WB																							
addi \$14, \$14, 4					IF	ID	EX	M	WB																						
lw \$2, 0(\$14)						IF			ID	EX	M	WB																			
add \$2, \$2, \$11									IF			ID	EX	M	WB																
beq \$1, \$2,												IF			ID	EX	M	WB													
addi \$14, \$14, 4																	IF	ID	EX	M	WB										
lw \$3, 4(\$14)																		IF			ID	EX	M	WB							
sub \$1, \$1, \$3																					IF			ID	EX	M	WB				
sw \$1, 8(\$14)																						IF			ID	EX	M	WB			

b) Manipule el código para reducir el número de ciclos en que se ejecuta el código en el mismo procesador del apartado anterior. Analice su ejecución e indique en cuántos ciclos se ejecuta ahora. Recuerde que el programa propuesto deberá ser igual de efectivo que el del enunciado.

Dado que el procesador no incorpora ninguna técnica de optimización, podemos reordenar el código. Además, un análisis en profundidad del mismo nos llevará a eliminar instrucciones redundantes, puesto que vemos que tras la instrucción de salto condicional se ejecutan en ambas ramas del salto las instrucciones addi \$14, \$14, 4 (que moveremos a la parte del código anterior al salto condicional) y sub \$1, \$1, \$3 (que moveremos a la parte correspondiente a la etiqueta SEGUIR). El código queda como sigue:

```

lw $1, 0($14)
addi $14, $14, 4
lw $2, 0($14)
add $1, $1, $10
addi $14, $14, 4
add $2, $2, $11
beq $1, $2, ETIQUETA
lw $3, 0($14)
j SEGUIR
ETIQUETA: lw $3, 4($14)
SEGUIR:   sub $1, $1, $3
          sw $1, 8($14)

```


1) Si el salto NO se toma:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
lw \$1, 0(\$14)	IF	ID	EX	M	WB																						
addi \$14, \$14, 4		IF	ID	EX	M	WB																					
lw \$2, 0(\$14)			IF			ID	EX	M	WB																		
add \$1, \$1, \$10						IF	ID	EX	M	WB																	
addi \$14, \$14, 4							IF	ID	EX	M	WB																
add \$2, \$2, \$11								IF	ID	EX	M	WB															
beq \$1, \$2									IF			ID	EX	M	WB												
lw \$3, 0(\$14)															IF	ID	EX	M	WB								
j SEGUIR																IF	ID	EX	M	WB							
sub \$1, \$1, \$3																				IF	ID	EX	M	WB			
sw \$1, 8(\$14)																					IF			ID	EX	M	WB

2) El salto se toma:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
lw \$1, 0(\$14)	IF	ID	EX	M	WB																				
addi \$14, \$14, 4		IF	ID	EX	M	WB																			
lw \$2, 0(\$14)			IF			ID	EX	M	WB																
add \$1, \$1, \$10						IF	ID	EX	M	WB															
addi \$14, \$14, 4							IF	ID	EX	M	WB														
add \$2, \$2, \$11								IF	ID	EX	M	WB													
beq \$1, \$2									IF			ID	EX	M	WB										
lw \$3, 4(\$14)															IF	ID	EX	M	WB						
sub \$1, \$1, \$3																IF			ID	EX	M	WB			
sw \$1, 8(\$14)																			IF			ID	EX	M	WB

c) En este caso el procesador segmentado incorpora ranura de retardo de tamaño 1. Prepare el código propuesto en el apartado b) para su ejecución en este procesador en el menor número posible de ciclos. Justifique en todo momento sus decisiones indicando en cuántos ciclos se ejecuta.

Nota: La instrucción **addi** se tratará a todos los efectos como una instrucción **add**. La instrucción de salto incondicional **j** se tratará a todos los efectos como una instrucción de salto condicional.

Tal cual ha quedado, el código propuesto en el apartado b) se podría ejecutar en el procesador con ranura de retardo de tamaño 1. Sin embargo, aprovechando esta técnica de optimización, se pueden introducir ciertos cambios que reducen aún más el número de ciclos en que se ejecuta el código. El código que se propone en este caso es el siguiente:

```

lw $1, 0($14)
addi $14, $14, 4
lw $2, 0($14)
add $1, $1, $10
addi $14, $14, 4
add $2, $2, $11
beq $1, $2, ETIQUETA
lw $3, 0($14)
                lw $3, 4($14)
ETIQUETA:  sub $1, $1, $3
                sw $1, 8($14)

```

1) Si el salto no se toma:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
lw \$1, 0(\$14)	IF	ID	EX	M	WB																				
addi \$14, \$14, 4		IF	ID	EX	M	WB																			
lw \$2, 0(\$14)			IF			ID	EX	M	WB																
add \$1, \$1, \$10						IF	ID	EX	M	WB															
addi \$14, \$14, 4							IF	ID	EX	M	WB														
add \$2, \$2, \$11								IF	ID	EX	M	WB													
beq \$1, \$2									IF			ID	EX	M	WB										
lw \$3, 0(\$14)												IF	ID	EX	M	WB									
lw \$3, 4(\$14)															IF	ID	EX	M	WB						
sub \$1, \$1, \$3																IF			ID	EX	M	WB			
sw \$1, 8(\$14)																		IF				ID	EX	M	WB

2) Si el salto se toma:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
lw \$1, 0(\$14)	IF	ID	EX	M	WB																	
addi \$14, \$14, 4		IF	ID	EX	M	WB																
lw \$2, 0(\$14)			IF			ID	EX	M	WB													
add \$1, \$1, \$10						IF	ID	EX	M	WB												
addi \$14, \$14, 4							IF	ID	EX	M	WB											
add \$2, \$2, \$11								IF	ID	EX	M	WB										
beq \$1, \$2									IF			ID	EX	M	WB							
lw \$3, 0(\$14)												IF	ID	EX	M	WB						
sub \$1, \$1, \$3															IF	ID	EX	M	WB			
sw \$1, 8(\$14)																IF			ID	EX	M	WB

Problema 15

Sea el siguiente fragmento de código:

Bucle:

```

lw $1, 0($2)
addi $1, $1, 1
sw $1, 0($2)
addi $2, $2, 4
sub $4, $3, $2
bne $4, Bucle

```

para el que inicialmente se considerará que el contenido del registro \$3 es $\$3 = \$2 + 396$. Se recuerda que bne es una pseudoinstrucción en la que se salta a la instrucción destino indicada por la etiqueta si el contenido del registro indicado es distinto de cero. Se pide:

- a) Esquema de ejecución del código si se ejecuta en el MIPS segmentado sin ningún tipo de optimización (es decir, sin adelantamiento, sin ranuras de retardo y sin predicción de salto), pero permitiéndose escribir y leer en el banco de registros en el mismo ciclo de reloj; además, los saltos se gestionarán vaciando el camino de datos. Calcular también el número de ciclos de reloj que se invierten en su ejecución. **(1 PUNTO)**

El esquema de ejecución, teniendo en cuenta las dependencias entre las instrucciones, es el siguiente:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
i1	IF	ID	EX	M	WB													
i2		IF			ID	EX	M	WB										
i3					IF			ID	EX	M	WB							
i4								IF	ID	EX	M	WB						
i5									IF			ID	EX	M	WB			
i6												IF			ID	EX	M	WB
																		IF

Tras la ejecución de la instrucción i6, se debe vaciar el cauce hasta que se pueda tomar la decisión de saltar o no. Esto se puede saber en el ciclo 18.

Por otro lado, como se comienza con $\$3 = \$2 + 396$, puesto que se va sumando 4 al valor de $\$2$, el bucle se ejecutará 99 veces. De ellas, 98 veces se volverá a saltar a la instrucción i1, que forma parte del bucle, mientras que en la última iteración se ejecutará una instrucción ajena a este código.

Así, en las 98 primeras iteraciones, tras el salto comienza a ejecutarse la instrucción i1, con lo que se invierten 17 ciclos en ejecutar todo el bucle. En la última iteración se consumirán los 18 ciclos.

Por lo tanto: $n^\circ \text{ ciclos} = 98 \times 17 + 18 = 1684$

- b) Suponer que el procesador MIPS segmentado tiene implementada la técnica de adelantamiento. ¿Cuántos ciclos se invertirá en este caso? **(1 PUNTO)**

Si se aplica la técnica de adelantamiento o anticipación, el esquema de ejecución es el siguiente:

	1	2	3	4	5	6	7	8	9	10	11
i1	IF	ID	EX	M	WB						
i2		IF		ID	EX	M	WB				
i3				IF	ID	EX	M	WB			
i4					IF	ID	EX	M	WB		
i5						IF	ID	EX	M	WB	
i6							IF	ID	EX	M	WB
											IF

Por idénticas razones a las del apartado anterior, $n^\circ \text{ ciclos} = 98 \times 10 + 11 = 991$

- c) Si además de adelantamiento, el procesador MIPS segmentado tiene implementada la técnica de ranura de retardo de tamaño 1, reordenar las instrucciones y modificar los operandos necesarios para que el código se ejecute en el menor número posible de ciclos. **(1 PUNTO)**

El siguiente código elimina las dependencias de datos a la vez que utiliza correctamente la ranura de retardo de tamaño 1 que, según el enunciado de este apartado, implementa el procesador. Se debe observar que no se han añadido ni eliminado instrucciones y que este código hace exactamente lo mismo que el original.

Bucle:

```
lw $1, 0($2)
addi $2, $2, 4
addi $1, $1, 1
sub $4, $3, $2
bne $4, Bucle
sw $1, -4($2)
```